

50-Day Python DSA Roadmap

A day-by-day plan to build strong Data Structures & Algorithms skills in Python — from fundamentals to interview-ready problem solving, organized topic by topic.

Roadmap Overview

Phase	Days	Focus Area
Phase 1: Foundations	1 – 7	Python refresher, complexity analysis, arrays
Phase 2: Core Linear Structures	8 – 17	Strings, searching, sorting, linked lists, stacks
Phase 3: Queues, Recursion & Backtracking	18 – 26	Queues, recursion, backtracking, divide & conquer
Phase 4: Trees	27 – 34	Binary trees, BSTs, tries, segment trees
Phase 5: Heaps & Graphs	35 – 43	Heaps, BFS/DFS, shortest paths, union-find
Phase 6: Dynamic Programming	44 – 48	Memoization, tabulation, classic DP patterns
Phase 7: Consolidation & Mock Practice	49 – 50	Mixed review and timed mock interviews

Phase 1: Foundations

Days 1 – 7

Day	Topic	Focus
1	Python Refresher	Lists, tuples, dicts, sets, slicing, comprehensions, *args/**kwargs
2	Time & Space Complexity	Big-O, Big-Ω, Big-Θ; analyze simple code snippets
3	Math for DSA	GCD/LCM, prime sieve, modular arithmetic, fast exponentiation
4	Bit Manipulation I	AND/OR/XOR/shifts, check power of 2, count set bits
5	Bit Manipulation II	Subsets via bitmask, single number problems
6	Arrays I	Traversal, prefix sums, two-pointer technique
7	Arrays II	Sliding window (fixed & variable size)

PRACTICE PROBLEMS

Two Sum, Best Time to Buy/Sell Stock, Max Subarray (Kadane's), Move Zeroes, Product of Array Except Self.

Phase 2: Core Linear Structures

Days 8 – 17

Day	Topic	Focus
8	Strings I	Palindrome checks, anagrams, string builder patterns
9	Strings II	Sliding window on strings (longest substring problems)
10	Searching	Linear search, Binary search (iterative + recursive)
11	Binary Search Variants	Search in rotated array, first/last occurrence, search on answer
12	Sorting I	Bubble, Selection, Insertion sort — implement from scratch
13	Sorting II	Merge sort, Quick sort — implement + complexity analysis
14	Linked List I	Singly linked list from scratch: insert, delete, traverse
15	Linked List II	Reverse a linked list, detect cycle (Floyd's), find middle
16	Linked List III	Merge two sorted lists, remove Nth node, doubly linked list
17	Stacks	Implement stack, valid parentheses, next greater element

PRACTICE PROBLEMS

Reverse Linked List, Linked List Cycle, Merge Two Sorted Lists, Valid Parentheses, Min Stack.

Phase 3: Queues, Recursion & Backtracking

Days 18 – 26

Day	Topic	Focus
18	Queues	Implement queue, circular queue, deque
19	Stack/Queue Applications	Implement queue using stacks & vice versa
20	Recursion I	Base cases, call stack visualization, factorial/fibonacci
21	Recursion II	Recursion on arrays/strings (reverse, subsequences)
22	Backtracking I	Permutations, combinations
23	Backtracking II	Subsets, N-Queens
24	Backtracking III	Sudoku solver, word search
25	Divide & Conquer	Merge sort revisited, majority element, closest pair concept
26	Review + Contest	Solve a timed set of 5 mixed problems from Phases 1–3

PRACTICE PROBLEMS

Generate Parentheses, Permutations, Subsets, Combination Sum, N-Queens.

Phase 4: Trees

Days 27 – 34

Day	Topic	Focus
27	Binary Trees I	Node structure, traversals (inorder/preorder/postorder) — recursive
28	Binary Trees II	Level order traversal (BFS), tree height, diameter
29	Binary Trees III	Iterative traversals using stacks
30	Binary Search Trees I	Insert, search, delete in BST
31	Binary Search Trees II	Validate BST, kth smallest, LCA in BST
32	Tree Problems	Lowest Common Ancestor (general tree), path sum problems
33	Tries	Implement Trie, insert/search/startsWith
34	Segment Trees / Fenwick (intro)	Range sum queries — conceptual + basic implementation

PRACTICE PROBLEMS

Validate BST, LCA of Binary Tree, Diameter of Binary Tree, Implement Trie, Path Sum II.

Phase 5: Heaps & Graphs

Days 35 – 43

Day	Topic	Focus
35	Heaps I	Min-heap/max-heap using heapq, implement heap from scratch
36	Heaps II	Kth largest element, top-K frequent elements
37	Heaps III	Merge K sorted lists, median finder (two heaps)
38	Graphs I	Representations (adjacency list/matrix), BFS
39	Graphs II	DFS (recursive + iterative), connected components
40	Graphs III	Topological sort (Kahn's + DFS-based), cycle detection
41	Graphs IV	Dijkstra's algorithm (shortest path)
42	Graphs V	Union-Find (Disjoint Set), Kruskal's MST
43	Graphs VI	Number of islands, clone graph, bipartite check

PRACTICE PROBLEMS

Course Schedule, Number of Islands, Clone Graph, Network Delay Time, Kth Largest Element.

Phase 6: Dynamic Programming

Days 44 – 48

Day	Topic	Focus
44	DP Foundations	Memoization vs tabulation, Fibonacci, climbing stairs
45	1D DP	House robber, coin change, longest increasing subsequence
46	2D DP I	Unique paths, minimum path sum, edit distance
47	2D DP II	Longest common subsequence, 0/1 knapsack
48	DP on Strings/Subsequences	Palindromic substrings, word break

PRACTICE PROBLEMS

Climbing Stairs, Coin Change, LIS, Edit Distance, 0/1 Knapsack, Word Break.

Phase 7: Consolidation & Mock Practice

Days 49 – 50

Day	Topic	Focus
49	Full Mixed Review	Timed practice set (6–8 problems spanning arrays, trees, graphs, DP). Review weak areas identified.
50	Mock Interview Simulation	2 problems in 45 minutes each, explain approach out loud, analyze complexity, review solutions. Redo any problem you struggled with.

PRACTICE PROBLEMS

Repeat your hardest problem from each phase without looking at your old solution.